

Unit-02

Basics of C#

2.1. Introduction to C#

C# is a modern, object-oriented programming language developed by Microsoft as part of the .NET framework. It is designed for building a wide range of applications, from desktop software to web applications, mobile apps, and more. C# combines the efficiency of C++ with the simplicity of Visual Basic, making it ideal for developing Windows applications and enterprise solutions.

2.2. Data Types, Operators, Variables

- **Data Types:** in C# define the type of data that a variable can store. Each data type determines the kind of values the variable can hold and the operations that can be performed on those values. In C#, data types can be broadly categorized into two main types: Value Types and Reference Types.
 - **int:** Integer numbers (e.g., 10, -5)
 - **float:** Floating-point numbers (e.g., 3.14, -0.5)
 - **double:** Double precision floating-point numbers (e.g., 3.14159)
 - **char:** Single characters (e.g., 'A')
 - **string:** A sequence of characters (e.g., "Hello")
 - **bool:** Boolean values (true or false)
- **Operators:** An operator in C# is a symbol or keyword used to perform operations on variables and values. Operators allow you to manipulate data and variables in various ways. C# provides several types of operators, each serving different purposes.
 - **Arithmetic Operators:** +, -, *, /, %
 - **Relational Operators:** ==, !=, <, >, <=, >=
 - **Logical Operators:** &&, ||, !
 - **Assignment Operator:** =
 - **Increment/Decrement Operators:** ++, --
- **Variables:** A variable in C# is a storage location in memory that holds a value. The value of a variable can change during the execution of a program. Every variable in C# has a

data type, which determines the kind of data it can store (e.g., integers, floating-point numbers, strings, etc.).

2.3. Control Statements

Control Statements in C# are used to control the flow of execution in a program. They allow you to make decisions, repeat actions, or alter the program's flow based on conditions. The main types of control statements in C# are:

1. Conditional Statements

These are used to perform actions based on certain conditions.

a) if Statement

The if statement allows you to execute a block of code only if a specified condition is true.

b) if-else Statement

If the condition is false, the else block is executed.

c) else-if Ladder

This allows you to check multiple conditions.

d) switch Statement

The switch statement is used to execute one out of multiple blocks of code based on the value of a variable or expression.

2. Looping Statements

These statements allow you to execute a block of code multiple times based on a condition.

a) for Loop

The for loop is used when you know how many times you need to repeat a block of code.

b) while Loop

The while loop is used when you want to repeat a block of code as long as a condition is true.

c) do-while Loop

The do-while loop guarantees that the block of code will run at least once, even if the condition is false.

3. Jump Statements

Jump statements alter the normal flow of control in a program by skipping over code or exiting loops.

a) break Statement

The break statement is used to exit a loop or switch statement early.

b) continue Statement

The continue statement skips the current iteration of a loop and proceeds to the next iteration.

c) return Statement

The return statement is used to exit a method and optionally return a value.

4. The goto Statement

The goto statement is used to transfer control to another part of the program, often used with labels.

2.4 Arrays, Classes, Structures, and Enumerations in C#

These are fundamental building blocks in C# used to store, organize, and manage data in different forms.

1. Arrays

An array is a data structure that can store multiple values of the same data type. It provides a way to organize large amounts of data and access them efficiently.

Declaration and Initialization:

- To declare an array, you specify the data type followed by square brackets ([]).
- You can initialize an array at the time of declaration or later.

2. Classes

A class in C# is a blueprint for creating objects. It defines properties, methods, and behaviours that the objects created from the class will have.

Class Components:

- **Fields:** Variables that store data.
- **Methods:** Functions that define the behaviour of the class.
- **Constructor:** A special method used to initialize objects.
- **Access Modifiers:** Control access to class members (e.g., public, private).

3. Structures (Structs)

A structure is similar to a class, but it is a value type (while a class is a reference type). Structures are typically used to store small groups of related data.

4. Enumerations (Enums)

An enumeration (Enum) is a distinct type that defines a set of named constants. It provides a way to assign names to integral values to make code more readable and maintainable.

Differences Between Arrays, Classes, Structures, and Enumerations:

Feature	Array	Class	Structure	Enumeration
Type	Reference type	Reference type	Value type	Value type
Purpose	Stores multiple elements	Defines object blueprint	Stores small data groups	Defines named constants
Memory	Stored in heap	Stored in heap	Stored in stack	Stored in stack
Inheritance	No inheritance	Supports inheritance	Does not support inheritance	No inheritance
Default Values	Default values for each type	Default is null for objects	No default values	Assigned default values
Use Case	Storing similar data	Object-oriented design	Lightweight, small data	Working with constants

2.5. Partial Classes, Static Classes, Sealed Classes in C#

These three types of classes are special variations in C# that allow you to define the structure and behaviour of your classes in different ways. Here's a detailed explanation:

1. Partial Classes

A partial class allows you to split the definition of a class across multiple files. This is particularly useful when working with large classes or when generating code automatically (like in auto-generated code by tools or designers).

Syntax:

```
public partial class MyClass  
{  
    public void Method1() { }  
}
```

2. Static Classes

A static class in C# is a class that cannot be instantiated (you can't create objects of a static class). All members of a static class must be static as well, meaning they belong to the class itself rather than to instances of the class.

Syntax:

```
public static class MathUtility
{
    public static int Add(int a, int b) { return a + b; }

    public static int Multiply(int a, int b) { return a * b; }
}
```

3. Sealed Classes

A sealed class is a class that cannot be inherited. Sealing a class means that no other class can derive from it. This is useful when you want to prevent inheritance for certain classes to preserve functionality or avoid unintended behaviour.

```
public sealed class Car
{
    public void Start() { }
}
```

Comparison between Partial Classes, Static Classes and Sealed Classes:

Feature	Partial Class	Static Class	Sealed Class
Inheritance	Can be inherited	Cannot be inherited	Cannot be inherited
Instantiable	Yes	No	Yes
Members	Can have instance and static members	Must have static members only	Can have instance members
Purpose	Split class across files	Utility classes, helper methods	Prevent inheritance, final behaviour
Constructor	Can have constructors	Cannot have instance constructors	Can have constructors
Usage	Large classes, auto-generated code	Grouping utility functions	Finalizing behaviour in a class

2.6. Constructors and Destructors in C#

In C#, constructors and destructors are special methods used for object initialization and cleanup.

1. Constructor

A constructor is a special method that is automatically called when an object of a class is created. It is used to initialize the object's state (fields, properties, etc.) when an instance of the class is created.

- Constructors have the same name as the class.
- They don't have a return type (not even void).
- Constructors can be parameterized or non-parameterized.

Syntax:

```
public class Car
{
    // Constructor
    public Car()
    {
        Console.WriteLine("Car object created!");
    }
}
```

2. Destructor

A destructor is a special method that is automatically called when an object is destroyed. It is used for cleanup operations, such as releasing unmanaged resources. In C#, destructors are not explicitly called but are automatically invoked by the garbage collector when the object is no longer referenced.

- Destructors do not take parameters.
- Destructors are used mainly for resource cleanup, such as releasing unmanaged resources (e.g., file handles, database connections).
- In C#, destructors are implemented using a tilde (~) followed by the class name.
- C# does not guarantee when a destructor will be called since it depends on the garbage collector.

Syntax:

```
public class Car
```

```

{
    // Destructor

    ~Car()
    {
        Console.WriteLine("Car object is being destroyed!");
    }
}

```

Example: Constructor and Destructor in Action

```

public class Car
{
    public string Make;
    public string Model;

    // Constructor
    public Car(string make, string model)
    {
        Make = make;
        Model = model;

        Console.WriteLine($"Car {Make} {Model} created!");
    }

    // Destructor
    ~Car()
    {
        Console.WriteLine($"Car {Make} {Model} is being destroyed!");
    }
}

// Using the class

```

```

class Program
{
    static void Main()
    {
        Car car1 = new Car("Toyota", "Corolla");

        // Destructor will be called automatically when the object goes out of scope (or is garbage
        collected).

    }
}

```

2.7. Concept and Implementations of Inheritance and Polymorphism in C#

1. Inheritance

Inheritance is one of the core concepts in Object-Oriented Programming (OOP) where a class (derived class) inherits members (fields, methods, properties) from another class (base class). It allows code reusability and establishes a relationship between classes.

Inheritance Implementation

In this implementation, a derived class inherits from a base class and can access all public and protected members of the base class.

```

// Base class

public class Animal
{
    public string Name { get; set; }

    public void Eat()
    {
        Console.WriteLine($"{Name} is eating.");
    }
}

// Derived class

public class Dog : Animal

```



```

{

    public void Bark()

    {

        Console.WriteLine($"{Name} is barking.");

    }

}

class Program

{

    static void Main(string[] args)

    {

        Dog dog = new Dog();

        dog.Name = "Buddy";

        dog.Eat(); // Inherited from Animal class

        dog.Bark(); // Defined in Dog class

    }

}

```

2. Polymorphism

Polymorphism is another fundamental OOP concept that allows objects of different types to be treated as objects of a common base type. The two main types of polymorphism are:

1. **Compile-time Polymorphism** (Static Binding) - Achieved through method overloading and operator overloading.
2. **Run-time Polymorphism** (Dynamic Binding) - Achieved through method overriding.

Method Overloading (Compile-time Polymorphism):

Overloading occurs when you define multiple methods with the same name but different parameters (either number, type, or both).

```

public class Calculator

{

    // Method overloading

```

```

public int Add(int a, int b)
{
    return a + b;
}

public double Add(double a, double b)
{
    return a + b;
}
}

class Program
{
    static void Main(string[] args)
    {
        Calculator calc = new Calculator();

        Console.WriteLine(calc.Add(3, 4));

        Console.WriteLine(calc.Add(2.5, 3.7));
    }
}

```

Method Overriding (Run-time Polymorphism):

Overriding occurs when a derived class provides its own implementation of a method defined in the base class.

- The method in the base class must be marked with the virtual keyword, and the method in the derived class must use the override keyword.

```

public class Animal
{
    public virtual void MakeSound()
    {

```

```
        Console.WriteLine("Animal makes a sound");
    }
}
```

```
public class Dog : Animal
{
    public override void MakeSound()
    {
        Console.WriteLine("Dog barks");
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        Animal myAnimal = new Animal();
        myAnimal.MakeSound();

        Animal myDog = new Dog();
        myDog.MakeSound();
    }
}
```

2.8. Concept and Implementation of Interfaces in C#

1. What is an Interface?

An interface in C# is a contract that defines a set of methods and properties without providing their implementation. Any class or struct that implements an interface must provide the implementation for all its methods and properties. Interfaces are used to achieve abstraction and polymorphism.

- An interface contains only the method signatures, properties, events, or indexers.
- It does not contain any implementation of the methods.
- A class or struct that implements an interface must provide the implementation for all members of the interface.
- A class can implement multiple interfaces.
- Interfaces can be used to define behavior that can be shared across different classes.

Syntax:

```
public interface IAnimal  
  
{  
  
    void Speak(); // Method signature, no implementation  
  
    void Eat();   // Another method signature  
  
}
```

Implementing an Interface:

```
public class Dog : IAnimal  
  
{  
  
    public void Speak()  
  
    {  
  
        Console.WriteLine("Dog barks");  
  
    }  
  
    public void Eat()  
  
    {  
  
        Console.WriteLine("Dog is eating");  
  
    }  
  
}
```

Advantages of Using Interfaces:

- 1) Abstraction

- 2) Multiple Inheritance
- 3) Loose Coupling
- 4) Code Reusability
- 5) Flexibility
- 6) Separation of Concerns
- 7) Interoperability

2.9. Virtual Methods, Abstract Classes, and Methods

1. Virtual Methods

A virtual method is a method in a base class that can be overridden in a derived class. It provides a default implementation but allows derived classes to provide their own specific implementation by overriding it.

- Declared using the virtual keyword.
- Can be overridden in derived classes using the override keyword.
- Provides a default implementation that can be replaced in derived classes.

Syntax:

```
public class BaseClass
{
    public virtual void Speak()
    {
        Console.WriteLine("Base class speaking");
    }
}

public class DerivedClass : BaseClass
{
    public override void Speak()
    {
        Console.WriteLine("Derived class speaking");
    }
}
```

2. Abstract Classes

An abstract class is a class that cannot be instantiated on its own. It is used to provide a common definition for derived classes. Abstract classes can contain abstract methods, concrete methods, and properties.

- Declared using the abstract keyword.
- Can contain abstract methods (without implementation).
- Can also contain regular methods (with implementation).
- Derived classes must implement the abstract methods.

Syntax:

```
public abstract class Animal
{
    public string Name { get; set; }

    public abstract void Speak();

    public void Eat()
    {
        Console.WriteLine($"{Name} is eating.");
    }
}

public class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Dog barks");
    }
}
```

3. Abstract Methods

An abstract method is a method defined in an abstract class without any body. It must be overridden by any non-abstract class that inherits from the abstract class.

- Declared using the abstract keyword.
- Do not have a body or implementation.
- Must be overridden in derived classes.

Syntax:

```
public abstract class Animal
{
    public abstract void MakeSound();
}

public class Dog : Animal
{
    public override void MakeSound()
    {
        Console.WriteLine("Woof!");
    }
}
```

Conclusion

- **Virtual Methods:** Allow overriding with a default implementation.
- **Abstract Classes:** Cannot be instantiated and can have both abstract and concrete methods.
- **Abstract Methods:** Must be implemented by derived classes, with no default implementation.

2.10. Exception Handling in C#

1. What is Exception Handling?

Exception handling in C# refers to the process of handling runtime errors (exceptions) that may occur during program execution. It helps to maintain the normal flow of the program even when unexpected errors occur.

Exception Handling Keywords in C#

1. **try:** Defines a block of code in which exceptions might occur.
2. **catch:** Catches exceptions thrown within the try block and handles them.
3. **finally:** Defines a block of code that runs after the try block, regardless of whether an exception occurred or not.
4. **throw:** Used to throw an exception intentionally.
5. **throw (rethrow):** Used to rethrow an exception after it has been caught in a catch block.

Program that demonstrates **Try-Catch Block**, **Try-Catch-Finally Block**, **Throwing an Exception**, and **Multiple Catch Blocks**:

```
using System;
```

```
class Program
```

```
{
```

```
    public static void ValidateAge(int age)
```

```
    {
```

```
        if (age < 18)
```

```
        {
```

```
            throw new ArgumentException("Age must be at least 18.");
```

```
        }
```

```
        Console.WriteLine("Age is valid.");
```

```
    }
```

```
    static void Main(string[] args)
```

```
    {
```

```
        try
```

```
        {
```

```
            int result = 10 / 0;
```

```
        }
```

```
        catch (DivideByZeroException ex)
```

```
        {
```

```
            Console.WriteLine("Error: " + ex.Message);
```

```
        }
```

```
        try
```

```
        {
```



```
int[] numbers = new int[2];

numbers[3] = 10;

}

catch (IndexOutOfRangeException ex)

{

    Console.WriteLine("Error: " + ex.Message);

}

finally

{

    Console.WriteLine("Execution completed.");

}

try

{

    ValidateAge(15);

}

catch (ArgumentException ex)

{

    Console.WriteLine("Error: " + ex.Message);

}

try

{

    int result = 10 / 0;

}

catch (DivideByZeroException ex)

{

    Console.WriteLine("Cannot divide by zero: " + ex.Message);
```

```
}  
catch (Exception ex)  
{  
    Console.WriteLine("General error: " + ex.Message);  
}  
}  
}
```